

SKILL – WEEK 3

Session 5:

1. To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases

```
sanjay_dundra@Sanjay:~/shell_project$ gcc -Wall -Wextra shell.c -o shell
sanjay_dundra@Sanjay:~/shell_project$ ./shell
=====
          SINGLE QUOTE PARSER SHELL
=====
Single quotes preserve literal content.
Type 'exit' to quit.

shell> echo 'hello world'

===== TOKENS =====
Token 1: echo
Token 2: hello world
=====

Syntax: VALID

===== PARSE TREE =====
COMMAND
  +-- echo
  +-- hello world
=====

===== EXECUTION STRUCTURE =====
Argument 1: echo
Argument 2: hello world
=====
```

Report: I tested the shell with a single-quoted string using echo 'hello world'. The parser correctly preserved the quoted content as one token, validated the syntax, and generated the correct parse tree and execution structure. Thus, I successfully verified single-quote handling in the shell.

```
shell> shell> echo '$HOME'

===== TOKENS =====
Token 1: shell>
Token 2: echo
Token 3: $HOME
=====

Syntax: VALID

===== PARSE TREE =====
COMMAND
  +-- shell>
  +-- echo
  +-- $HOME
=====

===== EXECUTION STRUCTURE =====
Argument 1: shell>
Argument 2: echo
Argument 3: $HOME
=====
```

Report: I tested the shell with echo '\$HOME'. The parser correctly preserved \$HOME as a single token without variable expansion, validated the syntax, and generated the correct parse tree and execution structure. Thus, I successfully verified single-quote handling and literal variable preservation in the shell.

```

shell> shell> echo 'hello | world'

===== TOKENS =====
Token 1: shell>
Token 2: echo
Token 3: hello | world
=====

Syntax: VALID

===== PARSE TREE =====
COMMAND
  +-- shell>
  +-- echo
  +-- hello | world
=====

===== EXECUTION STRUCTURE =====
Argument 1: shell>
Argument 2: echo
Argument 3: hello | world
=====

```

Report: I tested the shell with echo 'hello | world'. The parser correctly preserved the entire quoted string, including the pipe symbol, as a single token, validated the syntax, and generated the correct parse tree and execution structure. Thus, I successfully verified that special characters inside single quotes are treated as literal text.

```

shell> shell> echo 'this is a test'

===== TOKENS =====
Token 1: shell>
Token 2: echo
Token 3: this is a test
=====

Syntax: VALID

===== PARSE TREE =====
COMMAND
  +-- shell>
  +-- echo
  +-- this is a test
=====

===== EXECUTION STRUCTURE =====
Argument 1: shell>
Argument 2: echo
Argument 3: this is a test
=====

```

```
shell> echo 'hello world
Syntax Error: Unmatched single quote.
```

Report: I tested the shell with a valid single-quoted string and an unmatched single quote. The parser correctly treated the quoted text as one token and validated the first command, while the second command correctly detected an unmatched single quote and displayed a syntax error. Thus, I successfully verified both valid single-quote handling and error detection.

```
shell> ls | grep txt

===== TOKENS =====
Token 1: ls
Token 2: |
Token 3: grep
Token 4: txt
=====

Syntax: VALID

===== PARSE TREE =====
COMMAND
  +-- ls
  |
  +-- PIPE
  +-- grep
  +-- txt
=====

===== EXECUTION STRUCTURE =====
Argument 1: ls
Operator: PIPE
Argument 3: grep
Argument 4: txt
=====
```

Report: I tested the shell with the pipeline command `ls | grep txt`. The parser correctly identified `|` as a PIPE operator, validated the syntax, and generated the correct parse tree and execution structure for connecting `ls` output to `grep txt`. Thus, I successfully verified basic pipe handling in the shell.

2. To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands

```
=====
      QUOTED STRING PARSER SHELL
=====
Single quotes preserve literal text.
Double quotes preserve spaces and expand variables.
Type 'exit' to quit.

shell> echo "hello world"

===== TOKENS =====
Token 1: echo
Token 2: hello world
=====

Syntax: VALID

===== PARSE TREE =====
COMMAND
  +-- echo
  +-- hello world
=====

===== EXECUTION STRUCTURE =====
Argument 1: echo
Argument 2: hello world
=====
```

Report: I tested the shell with the double-quoted string echo "hello world". The parser correctly preserved the text as a single token, validated the syntax, and generated the correct parse tree and execution structure. Thus, I successfully verified double-quote handling and preservation of spaces within quoted strings.

```
shell> echo "$HOME"
```

```
===== TOKENS =====
```

```
Token 1: echo
```

```
Token 2: /home/sanjay_dundra@Sanjay
```

```
=====
```

```
Syntax: VALID
```



Report: I tested the shell with the double-quoted variable echo "\$HOME". The parser correctly expanded the \$HOME variable to /home/ajithmutyala, preserved it as a single token, and validated the syntax. Thus, I successfully verified variable expansion within double quotes.

```
shell> echo "Home directory is $HOME"
```

```
===== TOKENS =====
```

```
Token 1: echo
```

```
Token 2: Home directory is /home/sanjay_dundra@Sanjay
```

```
=====
```

```
Syntax: VALID
```



Report: I tested the shell with the double-quoted command echo "Home directory is \$HOME". The parser correctly expanded \$HOME to /home/ajithmutyala while preserving the complete text as a single token and validated the syntax. Thus, I successfully verified variable expansion within a double-quoted string.

```
shell> echo '$HOME'

===== TOKENS =====
Token 1: echo
Token 2: $HOME
=====

Syntax: VALID
```

Report: I tested the shell with the single-quoted command `echo '$HOME'`. The parser correctly preserved `$HOME` as literal text without expanding the variable and validated the syntax. Thus, I successfully verified that single quotes prevent variable expansion.

```
shell> echo "hello | world"

===== TOKENS =====
Token 1: echo
Token 2: hello | world
=====

Syntax: VALID
```

Report: I tested `echo "hello | world"`. The parser preserved the pipe (`|`) as literal text instead of treating it as a pipeline operator, producing two tokens: `echo` and `hello | world`. Syntax validated successfully. This confirms double quotes suppress special-character interpretation.

```
shell> echo "hello world" | grep hello

===== TOKENS =====
Token 1: echo
Token 2: hello world
Token 3: |
Token 4: grep
Token 5: hello
=====

Syntax: VALID
```

Report: I tested `echo "hello world" | grep hello`. The parser preserved `hello world` as a single quoted token but correctly recognized the unquoted pipe (`|`) as a pipeline operator, producing five tokens: `echo`, `hello world`, `|`, `grep`, `hello`. Syntax

validated successfully. This confirms the parser distinguishes literal characters inside quotes from active metacharacters outside them.

```
shell> echo "User: $USER Home:$HOME"

===== TOKENS =====
Token 1: echo
Token 2: User: sanjay_dundra@Sanjay Home: /home/sanjay_dundra
=====
Syntax: VALID
```

Report: I tested `echo "User: $USER Home: $HOME"`. The parser expanded both `$USER` and `$HOME` to their actual values (ajithmutyala and /home/ajithmutyala) while still preserving the surrounding text as a single token: `User: ajithmutyala Home: /home/ajithmutyala`. Syntax validated successfully. This confirms that, unlike single quotes, double quotes allow variable expansion while still grouping the content into one token.

Session 6:

1. To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs

```
shell> echo hello\ world

===== TOKENS =====
Token 1: echo
Token 2: hello world
=====

Syntax: VALID
```

Report: I tested `echo hello\ world`. The parser treated the escaped space (`\`) as a literal character rather than a token separator, merging `hello` and `world` into a single token: `hello world`. Syntax validated successfully. This confirms that a backslash escapes the following character, suppressing its special meaning (here, word-splitting on whitespace).


```
shell> echo hello\|world

===== TOKENS =====
Token 1: echo
Token 2: hello|world
=====

Syntax: VALID
```

Report: I tested echo hello\|world. The parser treated the escaped pipe (\|) as a literal character rather than a pipeline operator, merging it into a single token: hello|world. Syntax validated successfully. This confirms that a backslash suppresses the special meaning of metacharacters like |, not just whitespace.

```
shell> echo \$HOME

===== TOKENS =====
Token 1: echo
Token 2: $HOME
=====

Syntax: VALID
```

Report: I tested echo \\$HOME. The parser treated the escaped \$ as a literal character, preventing variable expansion and producing the token \$HOME (not the expanded path). Syntax validated successfully. This confirms that a backslash suppresses expansion by escaping the \$ sigil itself, similar to single-quote behavior but applied to just one character.

```
shell> echo hello\\world

===== TOKENS =====
Token 1: echo
Token 2: hello\world
=====

Syntax: VALID
```

Report: I tested echo hello\\world. The parser interpreted the double backslash (\\) as an escaped backslash, producing a single literal backslash in the output token: hello\\world. Syntax validated successfully. This confirms that a backslash can escape itself, allowing a literal \ to appear in the token.

```
shell> echo hello\  
Syntax Error: Incomplete escape sequence.
```

Report: I tested echo hello\ with a trailing backslash and no following character. The parser detected that the escape sequence had no character to apply to and correctly rejected it, reporting Syntax Error: Incomplete escape sequence. This confirms the parser validates that every backslash must be followed by a character to escape, rather than silently ignoring or misinterpreting a dangling backslash.

2. To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching

```
shell> ls  
  
===== TOKENS =====  
Token 1: ls  
=====  
Syntax: VALID  
  
===== PARSE TREE =====  
COMMAND  
  +-- ls  
=====  
===== EXECUTION STRUCTURE =====  
Argument 1: ls  
=====  
Launching command: ls  
Parent process waiting for child PID: 916  
Child process created. PID: 916  
Makefile  shell  shell.c  
Child exited with status: 0
```

Report: I tested ls. The parser produced a single token (ls), validated the syntax, and built a parse tree with ls as the sole command node. The shell then forked a child process (PID 916) to execute it, listing the directory contents (Makefile, shell, shell.c), and the child exited with status 0.

```

shell> ls -l
===== TOKENS =====
Token 1: ls
Token 2: -l
=====
Syntax: VALID
===== PARSE TREE =====
COMMAND
+--- ls
+--- -l
=====
===== EXECUTION STRUCTURE =====
Argument 1: ls
Argument 2: -l
=====
Launching command: ls
Parent process waiting for child PID: 921
Child process created. PID: 921
total 40
-rw-r--r-- 1 sanjay_dundra@Sanjay sanjay_dundra@Sanjay 48 Aug 12 08:31 Makefile
-rwxr-xr-x 1 sanjay_dundra@Sanjay sanjay_dundra@Sanjay 21184 Aug 13 16:52 shell
-rw-r--r-- 1 sanjay_dundra@Sanjay sanjay_dundra@Sanjay 11142 Aug 13 16:52 shell.c
Child exited with status: 0

```

Report: I tested `ls -l`. The parser produced two tokens (`ls`, `-l`), validated the syntax, and built a parse tree with `-l` as an argument node under the `ls` command. The shell forked a child process (PID 921) which executed `ls -l`, correctly displaying detailed file listings (permissions, owner, size, date) for `Makefile`, `shell`, and `shell.c`, then exited with status 0. This confirms the shell correctly handles commands with flags/arguments through the full parse-to-execution pipeline.

```

shell> pwd
===== TOKENS =====
Token 1: pwd
=====
Syntax: VALID
===== PARSE TREE =====
COMMAND
+-- pwd
=====
===== EXECUTION STRUCTURE =====
Argument 1: pwd
=====
Launching command: pwd
Parent process waiting for child PID: 922
Child process created. PID: 922
/home/ajithmutyala/shell_project
Child exited with status: 0

```

Report: I tested `pwd`. The parser produced a single token (`pwd`), validated the syntax, and built a parse tree with `pwd` as the sole command node. The shell forked a child process (PID 922) which executed `pwd`, correctly printing the current working directory (`/home/ajithmutyala/shell_project`), then exited with status 0. This confirms the shell correctly executes built-in-style commands with no arguments via `fork/exec`.

```

shell> echo "hello world"

===== TOKENS =====
Token 1: echo
Token 2: hello world
=====

Syntax: VALID

```

Report: I tested echo "hello world". The parser preserved the quoted string as a single token (hello world) rather than splitting it on the space, producing two tokens: echo and hello world. Syntax validated successfully. This confirms that double quotes group whitespace-separated words into one token.

```

shell> abczy

===== TOKENS =====
Token 1: abczy
=====

Syntax: VALID

===== PARSE TREE =====
COMMAND
  +-- abczy
=====

===== EXECUTION STRUCTURE =====
Argument 1: abczy
=====

Launching command: abczy
Parent process waiting for child PID: 933
Child process created. PID: 933
Execution failed: No such file or directory
Child exited with status: 1

```

Report: I tested abczy, an invalid command. Parsing succeeded normally, but execution failed with No such file or directory, and the child exited with status 1. This confirms the shell distinguishes syntax errors from runtime execution errors.